

Percolate Interview Assignment: GitHub Issue Triage Agent

Objective

Build a small system that monitors a GitHub repository for new and updated issues, triages them, and sends useful Slack notifications that help a team **prioritise, investigate, and close issues faster**.

The goal is not simply to pipe GitHub activity into Slack. The system should intelligently decide **what deserves attention, when, and how it should be communicated**.

Requirements

Your system should:

- Monitor a GitHub repository/account for new or updated issues.
- Analyse and triage each issue. At minimum, consider:
 - Severity / priority
 - Issue type
 - Likely action required
 - Whether the issue appears actionable
 - Whether it may duplicate or relate to another issue
- Send appropriate notifications to Slack.

Slack UX is an important part of the assignment

A naïve implementation that sends one Slack message per GitHub issue will create noise.

Design the notification strategy so that it:

- Surfaces urgent or high-value issues quickly.
- Avoids flooding Slack with low-priority activity.
- Organises issues for lowest friction and cognitive overhead.
- Gives enough context for someone to make a quick decision (or to dive deeper when necessary).
- Makes the next action obvious.
- Maintains enough transparency that the team understands what the system is filtering or deprioritising.

You may use rules, LLMs, embeddings, or any combination you think is appropriate.

Deliverables

Please submit:

1. A working prototype.
2. Source code with setup instructions.
3. A short README explaining:
 - Your architecture.
 - Your triage approach.
 - Your Slack notification strategy.
 - How you prevent notification overload.
 - Key assumptions and trade-offs.
4. A few example GitHub issues and the resulting Slack output screenshots.

What We Care About

We will primarily evaluate:

- **Product judgment:** Does this actually make an engineering team's life easier?
- **Signal vs. noise:** Are notifications thoughtful rather than mechanical?
- **Triage quality:** Does the system make sensible distinctions between issues?
- **Engineering quality:** Is the implementation clear and reasonably robust?
- **Decision transparency:** Can a human quickly understand why an issue was surfaced, grouped, or deprioritised?

Keep the scope small. We care more about a thoughtful end-to-end solution than production-scale infrastructure.